SmartSave — Personal Finance & Savings Application

A cross-platform mobile application for tracking expenses, analyzing spending habits, achieving savings goals, and building financial wellness.

---

1. Technology Stack

Layer	Technology	Details
Backend Runtime	Node.js (v26)	Express.js framework
Database	MongoDB	Mongoose ODM
Auth	JWT	Stateless token-based auth
Queue	Bull (Redis) + Bull Board UI	Background job processing
Real-time	Socket.io	Live notifications
AI Integration	OpenAI-compatible API	Financial advisor, OCR, recommendations
Frontend	Flutter/Dart	Cross-platform (iOS + Android)
State Management	Provider + ChangeNotifier	Per-feature provider classes
Architecture	Clean Architecture (Feature-first)	Domain/Data/Presentation layers
Infrastructure	Docker + Docker Compose	Containerized deployment

---

2. Project Structure

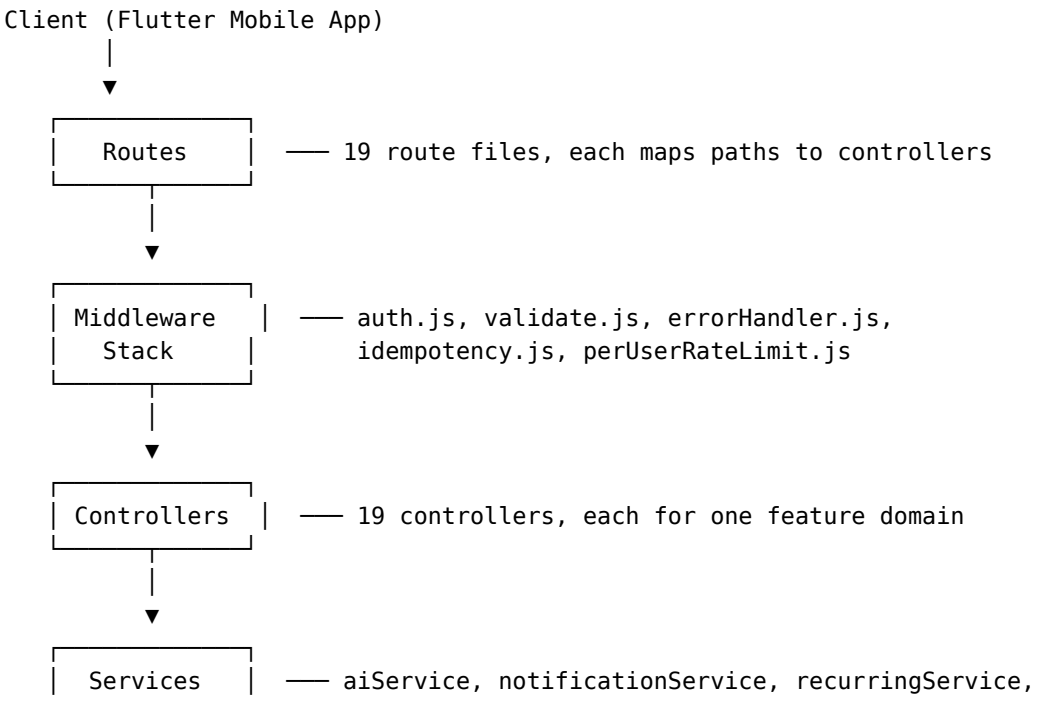
```
SmartSave/
├── backend/                          # Node.js/Express API
│   ├── src/
│   │   ├── config/                   # App configuration (DB, AI, env)
│   │   ├── controllers/              # Request handlers (19 controllers)
│   │   ├── middleware/               # Auth, validation, error handling, rate
limit
│   │   ├── models/                   # Mongoose schemas (15+ models)
│   │   ├── routes/                   # Express route definitions (19 route
files)
│   │   ├── services/                 # Business logic services (8 services)
│   │   ├── utils/                    # Helpers, audit, money, email, logger
│   │   └── server.js                 # Entry point
│   ├── Dockerfile
│   └── docker-compose.yml
│
```

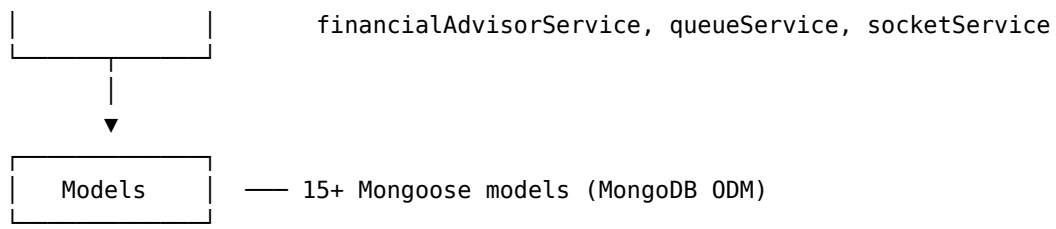
```
├── frontend/smartsave/              # Flutter mobile app
│   ├── lib/
│   │   ├── app/                     # App shell, routes, theme
│   │   ├── core/                    # DI, network client, constants, errors,
utilities
│   │   ├── data/                    # Data layer
│   │   │   ├── datasources/         # Remote (API) + Local (SQLite) datasources
│   │   │   ├── models/              # JSON-serializable DTOs
│   │   │   └── repositories/        # Repository implementations
│   │   ├── domain/                  # Business logic
│   │   │   ├── models/              # Domain entities
│   │   │   ├── repositories/        # Repository interfaces
│   │   │   └── usecases/            # Application-specific business rules
│   │   └── presentation/            # UI layer
│   │       ├── providers/           # State management (ChangeNotifiers)
│   │       ├── screens/             # 27 screens
│   │       └── widgets/             # Reusable UI components
│   ├── android/                     # Native Android configuration
│   ├── ios/                         # Native iOS configuration
│   └── pubspec.yaml
│
├── .github/                         # CI/CD workflows
└── README.md
```

---

3. Architecture Overview

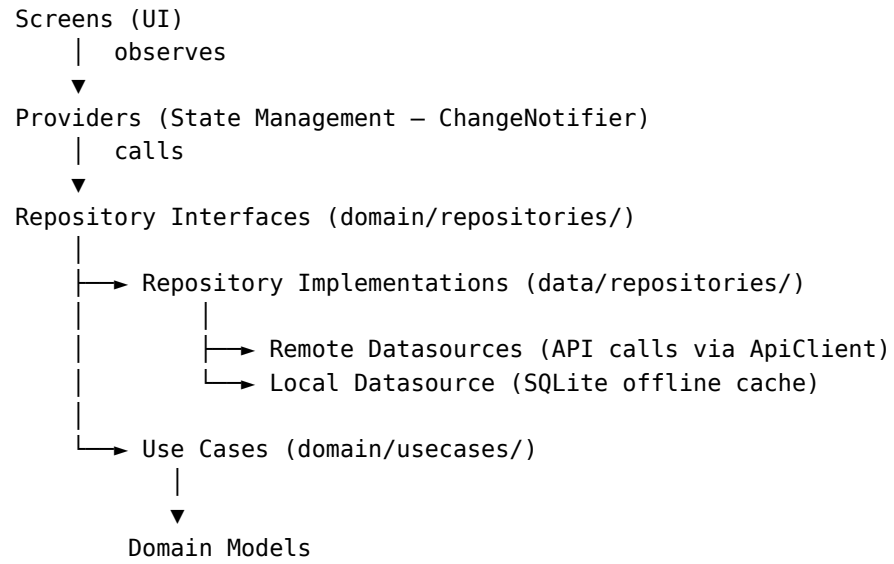
Backend Architecture (Node.js/Express)





Key architectural patterns: - **Middleware-based auth**: JWT verification happens in `middleware/auth.js` before any controller - **Async error handling**: `utils/catchAsync.js` wraps all controllers (19 imports — one of the most depended-on files) - **Idempotency**: `middleware/idempotency.js` prevents duplicate financial transactions - **Audit trail**: `utils/audit.js` logs all financial operations - **AI-powered features**: OpenAI integration for OCR, financial advice, and recommendations

Frontend Architecture (Flutter — Clean Architecture)



Key patterns: - **Offline-first**: Repositories try local cache first, fall back to remote API - **Dependency injection**: `core/di/service_locator.dart` wires the entire dependency graph - **Provider pattern**: Each feature has a dedicated ChangeNotifier provider

---

4. Backend API Endpoints

The backend exposes **19 route modules**, each with a corresponding controller:

Feature	Routes	Controller	Key Operations
Auth	`authRoutes.js`	`authController.js`	Login, register, forgot/reset password, profile
Transactions	`transactionRoutes.js`	`transactionController.js`	CRUD, filtering, categorization
Budget	`budgetRoutes.js`	`budgetController.js`	Monthly budget creation, tracking
Goals	`goalRoutes.js`	`goalController.js`	Savings goals, contributions, progress
Analytics	`analyticsRoutes.js`	`analyticsController.js`	Spending patterns, trends, insights
Calendar	`calendarRoutes.js`	`calendarController.js`	Financial calendar view
Recurring	`recurringRoutes.js`	`recurringController.js`	Recurring transaction templates
Auto-Save	`autoSaveRoutes.js`	`autoSaveController.js`	Automated savings rules
Net Worth	`netWorthRoutes.js`	`netWorthController.js`	Asset/liability tracking
Reports	`reportRoutes.js`	`reportController.js`	Exportable financial reports
Notifications	`notificationRoutes.js`	`notificationController.js`	In-app + push notifications
Challenges	`challengeRoutes.js`	`challengeController.js`	Financial wellness challenges
OCR	`ocrRoutes.js`	`ocrController.js`	Receipt scanning (AI-powered)
Financial Advisor	`financialAdvisorRoutes.js`	`financialAdvisorController.js`	AI financial advice
Recommendations	`recommendationRoutes.js`	`recommendationController.js`	Personalized suggestions
Export	`exportRoutes.js`	`exportController.js`	CSV/PDF export
XP/Gamification	`xpRoutes.js`	`xpController.js`	Streaks, achievements, levels
Subscriptions	`subscriptionRoutes.js`	`subscriptionController.js`	Subscription tracking
Profile	`profileRoutes.js`	`profileController.js`	User profile management

5. Data Models (Backend — Mongoose)

Core Financial Models

Model	Key Fields	Purpose
User	email, passwordHash, name, preferences, createdAt	User accounts and auth
Transaction	userId, amount, category, date, description, type (income/expense)	Financial transactions
Budget	userId, category, amount, month, year, spent	Monthly budget tracking
Goal	userId, name, targetAmount, currentAmount, deadline	Savings goals
RecurringTransaction	userId, amount, frequency, nextDate, template	Recurring bills/income
NetWorth	userId, assets[], liabilities[], computed total	Net worth tracking
AutoSave	userId, rules[], active	Automated savings

Gamification Models

Model	Key Fields	Purpose
Achievement	userId, badge, unlockedAt	Achievement badges
UserStreak	userId, currentStreak, longestStreak, lastActivity	Login/activity streaks
Challenge	userId, type, goal, progress, reward	Financial challenges

System Models

Model	Key Fields	Purpose
Notification	userId, type, title, body, read	In-app notifications
AuditLog	userId, action, resource, details, ip	Audit trail
AdvisorMemory	userId, context[], history	AI conversation memory
IdempotencyRequest	key, response, expiresAt	Idempotency support

6. Backend Services

Service	Responsibility
aiService	Generic AI integration (OpenAI-compatible API)
financialAdvisorService	Personal financial advice using AI with user context
notificationService	In-app + push notification dispatch
queueService	Bull/Redis job queue management
recurringService	Scheduled recurring transaction processing
redisService	Redis connection and caching
socketService	Socket.io real-time communication
aiContextBuilder	Builds AI prompt context from user financial data

7. Frontend UI Structure

Screens (27 screens)

Screen	Route	Purpose
splash_screen	/	App launch splash
onboarding_screen	/onboarding	First-time user onboarding
login_screen	/login	Authentication
register_screen	/register	Registration
forgot_password_screen	/forgot-password	Password recovery
dashboard_screen	/dashboard	Home — financial overview
transactions_screen	/transactions	All transactions list
add_expense_screen	/transactions/add-expense	New expense entry
add_income_screen	/transactions/add-income	New income entry
transaction_detail_screen	/transactions/:id	Transaction details
budget_screen	/budget	Budget management
goals_screen	/goals	Savings goals list
goal_detail_screen	/goals/:id	Goal progress detail
analytics_screen	/analytics	Spending analytics
calendar_screen	/calendar	Financial calendar
reports_screen	/reports	Financial reports
net_worth_screen	/net-worth	Asset/liability tracking
auto_save_screen	/auto-save	Auto-save rules
recurring_screen	/recurring	Recurring transactions
financial_advisor_screen	/financial-advisor	AI chat advisor
subscriptions_screen	/subscriptions	Subscription manager
challenges_screen	/challenges	Financial challenges
gamification_screen	/gamification	Achievements & levels
heatmap_screen	/heatmap	Spending heatmap
level_screen	/levels	XP progress
notifications_screen	/notifications	Notification history
profile_screen	/profile	User profile settings
settings_screen	/settings	App settings
quick_add_screen	/quick-add	Quick transaction entry

State Management Providers

Each feature has a dedicated ChangeNotifier provider: - `auth_provider` — Auth state, login/logout, token management - `transaction_provider` — Transaction CRUD, filtering - `budget_provider` — Budget tracking - `goal_provider` — Goal management - `analytics_provider` — Analytics data - `calendar_provider` — Calendar data - `notification_provider` — Notification state - `recurring_provider` — Recurring transactions - `auto_save_provider` — Auto-save rules - `net_worth_provider` — Net worth tracking - `subscription_provider` — Subscription management - `challenge_provider` — Challenge progress - `xp_provider` — XP and gamification - `report_provider` — Report generation - `financial_advisor_provider` — AI advisor chat - `theme_provider` — Theme/appearance settings

---

8. Dependency Graph (Key Relationships)

Most Imported Files (High Fan-in)

Imports	File	Why It's Central
19	`catchAsync.js`	Every async controller uses this wrapper
20	`auth.js`	Most routes require authentication middleware
14	`Transaction.js`	Central data model
11	`errorHandler.js`	Global error handler
11	`logger.js`	Logging across the app
11	`validate.js`	Request validation
9	`Budget.js`	Budget model
8	`Goal.js`	Goal model
7	`index.js` (config)	Centralized configuration
6	`User.js`	User model

Frontend Core Dependencies

Imports	File	Role
14	`provider`	State management library
14	`app_colors`	Theme colors used across all widgets
11	`api_client.dart`	HTTP client for all API calls
10	`auth_provider`	Auth state consumed by many screens
9	`currency_util`	Currency formatting shared everywhere

---

9. Cross-Cutting Concerns

Security

- **JWT authentication** via `middleware/auth.js`
- **Per-user rate limiting** via `middleware/perUserRateLimit.js`
- **Input validation** via `middleware/validate.js`
- **Idempotency** via `middleware/idempotency.js` (prevents duplicate financial operations)
- **Audit logging** via `utils/audit.js`

Error Handling

- Global error handler middleware catches all exceptions
- `catchAsync.js` wraps every async controller (eliminates try/catch boilerplate)
- Structured error responses with appropriate HTTP status codes

Background Jobs (Bull Queue)

- Redis-backed job queue for async processing (notifications, recurring tasks)
- Bull Board UI for job monitoring

Real-time Communication

- Socket.io for live updates (notifications, real-time balance changes)

AI Integration

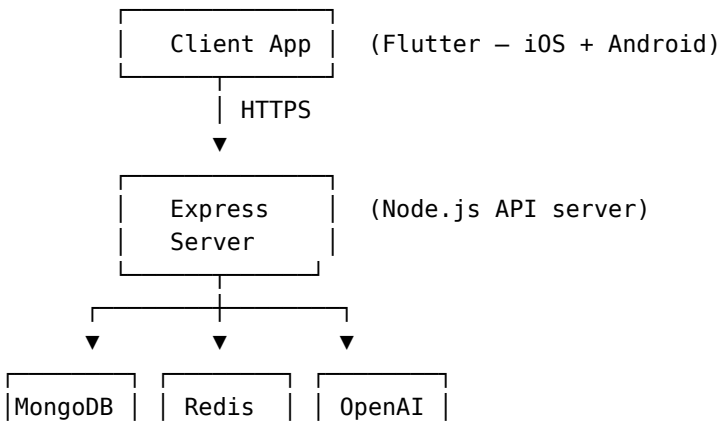
- **Financial Advisor**: Conversational AI that understands user's financial context
- **OCR Receipt Scanning**: AI-powered receipt text extraction
- **Recommendations**: AI-generated personalized financial suggestions

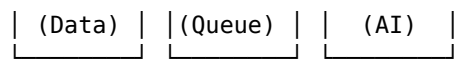
Offline Support (Frontend)

- Local SQLite database as offline cache
- Repository pattern: checks local data before fetching from API
- Remote datasources handle API calls with error fallback

---

10. Infrastructure & Deployment





- **Containerization**: Docker + Docker Compose for local dev and production
- **Process Management**: PM2 via `ecosystem.config.js` for production Node.js process management
- **CI/CD**: GitHub Actions workflows
- **Environment Configuration**: `.env.example` with documented configuration variables

---

11. Key Architectural Decisions

1. **MVC + Service Layer**: Controllers handle HTTP, services handle business logic, models handle data — clean separation of concerns.
2. **Clean Architecture (Frontend)**: Three-layer architecture (data/domain/presentation) ensures the UI is decoupled from data sources and business rules are testable.
3. **Feature-based Organization**: Both backend (controllers/routes per feature) and frontend (providers, screens, repos per feature) are organized by domain feature, not by technical layer.
4. **Monorepo-ish Structure**: Backend and frontend live in one repository with separate `package.json` / `pubspec.yaml` manifests.
5. **Provider over BLoC**: Flutter state management uses ChangeNotifier + Provider (not BLoC/Riverpod) — simpler and sufficient for the app's complexity.
6. **Idempotency by Design**: The `IdempotencyRequest` model and middleware prevent duplicate financial operations even under retries — critical for fintech correctness.
7. **Audit Trail**: Every financial operation is logged via `AuditLog` for full traceability and reconciliation.

---

12. Knowledge Graph Statistics

Metric	Count
Total Nodes	570
File Nodes	338 (79 backend, 147 frontend, 112 other)
Class Nodes	117
Function Nodes	77
Config Nodes	34
Service Nodes	2 (Docker)
Document Nodes	2
Total Edges	735
Import Edges	449
Contains Edges	148
Architecture Layers	8
Tour Steps	10

Architecture Layers

Layer	Files	Description
`backend-api`	38	HTTP route handlers and controllers for REST endpoints
`backend-data`	20	Mongoose models, DB config, server entry
`backend-services`	19	Middleware, services, utils
`backend-infrastructure`	9	Docker, root config, CI/CD
`frontend-presentation`	82	Screens and widgets (UI layer)
`frontend-data`	95	Data models, repositories, datasources
`frontend-core`	42	Providers, DI, network, constants
`frontend-platform`	71	Android/iOS native, build config, assets

Generated from knowledge graph analysis. Last updated: June 2026.